Cruthon: un lenguaje basado en teoría de conjuntos

Oct 20, 2026 · @Luis

Motivación

Desde los años noventa, la programación orientada a objetos (OOP) domina el desarrollo de software sin que haya surgido un reemplazo real. Lo que hubo fueron capas añadidas: funcional, reactiva, orientada a aspectos, orientada a actores. Todas conservan la misma idea de fondo: organizar el código en unidades con una identidad central y jerarquías de herencia.

Cruthon parte de una pregunta distinta: ¿qué pasa si el código se organiza con álgebra de conjuntos en vez de jerarquías? En vez de clases que heredan de un padre, Cruthon usa **blobs** (conjuntos) que se combinan mediante intersección, unión, diferencia y diferencia simétrica. La identidad de una entidad no viene de su posición en un árbol, sino de las operaciones de conjuntos en las que participa.

Conceptos base

- **Blob**: la unidad fundamental de Cruthon, equivalente a un conjunto. Reemplaza a la clase. Un archivo puede contener uno o varios blobs.
- **Intersección**: combinar dos blobs produce un blob nuevo que contiene solo lo que ambos comparten. Ese nuevo blob tiene memoria propia, compartida por los blobs que lo originaron.
- **Unión**: combina el espacio de ejecución completo de dos blobs sin exigir una zona común de datos, como correr ambos bajo un mismo proceso.
- **Diferencia**: un blob que parte del contenido de otro y excluye, en tiempo de definición, todo lo que también existe en un segundo blob.
- **Diferencia simétrica**: lo que no comparten dos blobs entre sí.
- **Inalcanzable**: un subconjunto dentro de un blob que no es producto de ninguna intersección. No es privado por diseño, simplemente no vive dentro de ninguna zona compartida, así que nada externo lo ve.

Anidamiento y visibilidad

Un blob puede contener subconjuntos propios. Si un subconjunto no fue creado como resultado de una intersección, los blobs de nivel superior no lo ven, por simple pertenencia geométrica, no por ninguna regla de acceso o encapsulamiento.

La regla general es: un blob contenedor ve todo lo que está dentro de su área, incluidas las intersecciones que contiene. Lo inalcanzable queda fuera de esa área por definición. No hay marcadores de privado o público, la visibilidad se deduce de dónde vive cada cosa, igual que en un diagrama de Venn: si estás dentro del área de intersección te ven, si no, no.

Semántica de memoria y ejecución

Operación	Qué pasa en memoria	Qué pasa en ejecución
Intersección	Se crea un blob nuevo con memoria propia, compartida por los blobs que lo originan (lectura y escritura por ambos)	Funciona como canal controlado: solo lo explícitamente puesto ahí se comparte
Unión	No crea memoria compartida nueva; combina el espacio de ejecución completo de ambos blobs	Equivale a correr dos programas bajo un mismo proceso, sin zona común de datos
Diferencia	Blob que parte del contenido completo del primero y excluye, en tiempo de definición, los nombres que también existen en el segundo	Ejecuta solo lo exclusivo de un blob respecto a otro
Diferencia simétrica	Memoria con lo que ninguno de los dos blobs comparte con el otro	Útil para aislar comportamiento único de cada blob

Pendiente de definir con precisión: el scope de una función llamada desde dentro de una intersección cuando quien llama pertenece solo a uno de los blobs originales. La propuesta de trabajo es que la zona de intersección tenga su propio estado aislado, sin heredar automáticamente el contexto de quien la llama, para evitar el problema clásico de variables globales mutables impredecibles.

Caso de uso: main, UI y backend

Un blob principal (main) contiene dos blobs hijos, UI y backend. Donde UI y backend se intersectan, se forma un tercer blob con funciones llamables y variables compartidas, el único punto de contacto entre ambos.

Esto equivale a paso de mensajes con memoria compartida acotada, pero expresado con álgebra de conjuntos en vez de sockets o pipes. UI y backend pueden seguir siendo casi completamente distintas entre sí; definir una intersección pequeña entre ellas basta como única zona de contacto controlado, nada se filtra que no esté explícitamente puesto ahí.

La misma idea se extiende a sistemas operativos: aislar procesos definiendo el contacto entre dos cosas como una forma geométrica (una intersección), en vez de como una lista de reglas o permisos de acceso.

Estrategia de implementación

Cruthon parte de un fork de Python en vez de un lenguaje desde cero, aprovechando su parser, su librería estándar y que los modelos de lenguaje ya lo entienden muy bien (precedente: proyectos como Cython o MicroPython, que adaptaron Python a un objetivo específico sin reescribirlo todo).

El cambio central está en el intérprete: reemplazar el orden de resolución de métodos (MRO), que es jerárquico, por resolución de pertenencia a conjuntos como mecanismo central del lenguaje. Python ya permite simular parte de esto con herencia múltiple, duck typing, protocolos y tipos de unión e intersección en las anotaciones estáticas, pero no es nativo: por dentro sigue resolviendo todo vía MRO.

Preguntas abiertas

- [ ] Scope resuelto: una función ejecuta siempre con el scope del blob donde está físicamente definida, nunca el de quien la llama. Confirmado en los ejemplos de carrito, unión y diferencia simétrica: el llamador solo entra por la puerta del blob de intersección o unión, pero adentro la función solo ve lo que vive en ese blob.
- [ ] Resuelto: el resultado de cualquier operación de conjuntos es en sí mismo un blob válido, y por lo tanto puede usarse como operando en otra operación (unión de una unión, intersección de dos intersecciones, etc.), sin límite de anidamiento. El conjunto vacío también es un resultado válido: un blob vacío es legal, simplemente no tiene miembros.

- [ ] Resuelto: no deberían compartir nada directamente, eso violaría la regla geométrica central. Si dos blobs sin intersección necesitan comunicarse, la solución es encadenar intersecciones como puente: si A y C no se tocan pero ambos se intersectan con B (A & B y B & C), entonces B actúa de intermediario sin que A y C necesiten verse directo. Es el equivalente a un bus de mensajes o capa intermedia, pero emerge naturalmente del álgebra de conjuntos en vez de ser un patrón aparte.
- [ ] Definir si las operaciones de conjuntos pueden variar según el tipo de operación que se está llevando a cabo, no solo según qué blobs participan
- [ ] Diseñar la sintaxis concreta del lenguaje (cómo se declaran blobs, intersecciones, uniones y diferencias en código)
- [ ] Prototipo mínimo, instrucciones para construirlo:
    1. Partir del código fuente oficial de CPython (repositorio github.com/python/cpython) como base y referencia, versión estable más reciente.
    2. Agregar la palabra clave `blob` al tokenizer y al parser (Grammar/python.gram), como un bloque equivalente a `class` pero sin herencia implícita.
    3. Implementar los cuatro operadores de conjuntos (`&`, `|`, `-`, `^`) a nivel de blobs en el compilador (Python/compile.c), generando una tabla de pertenencia por blob en vez de un MRO.
    4. Agregar el nuevo opcode `RESOLVE_MEMBER` a la máquina virtual (Python/ceval.c), que reemplaza `LOAD_ATTR` para accesos dentro de un blob: busca el nombre en la tabla de pertenencia del blob activo y en las intersecciones alcanzables.
    5. Mantener intacto todo el resto del pipeline (import de módulos, librería estándar, bytecode para código fuera de blobs) para heredar el ecosistema de Python sin cambios.
    6. Caso de prueba mínimo de éxito: compilar y ejecutar el ejemplo del carrito de compras de este documento, verificando que frontend no pueda acceder a db y que backend no pueda acceder a dibujar_pagina, mientras ambos leen y escriben correctamente en carrito.
    7. Crear un repositorio nuevo (fork de cpython o repositorio independiente que lo referencie), documentar las diferencias respecto a CPython estándar, e incluir los ejemplos de este documento como tests de integración.

Visualización y forma

Como las relaciones entre blobs son geométricas por diseño, en vez de jerárquicas, Cruthon se presta de forma natural a generar un mapa visual del código mientras corre o se compila, un diagrama de conjuntos en vivo que muestra qué blobs se tocan, se unen o se diferencian en cada momento. Esto es mucho más directo que en OOP, donde ese mapa hay que inferirlo reconstruyendo el árbol de herencia.

El nombre "blob" describe mejor la forma real de estas estructuras. Una vez que se encadenan uniones, diferencias e intersecciones, los blobs resultantes tienden a volverse formas irregulares, no esferas limpias.

Ejemplo de código: main, frontend y backend

Sintaxis propuesta e ilustrativa, no final (pendiente en la lista de preguntas abiertas). La intersección `carrito` es la única zona que frontend y backend comparten: guarda los items y el total, y ambos blobs pueden leerla y escribirla. Todo lo demás de cada blob es inalcanzable para el otro.

```
blob main:
    blob backend:
        db = conectar_base_datos()

        func calcular_envio(items):
            return db.consultar_tarifas(items)

        func validar_stock(item):
            return db.stock(item) > 0

    blob frontend:
        func dibujar_pagina(carrito):
            render("tienda.html", carrito.items, carrito.total)

        func click_comprar():
            mostrar_checkout()

    blob carrito = frontend & backend:
        items = []
        total = 0.0

        func agregar(item, precio):
            items.append(item)
            total += precio

    func iniciar():
        carrito.agregar("zapatos", 49.90)
        frontend.dibujar_pagina(carrito)
        backend.validar_stock("zapatos")
```

El backend nunca llama directo a `dibujar_pagina`, ni el frontend a `db`. Solo `carrito`, la intersección, es visible para ambos; el resto de cada blob queda fuera de esa área compartida.

Compilación: qué cambia respecto a CPython

CPython hoy compila el código fuente a bytecode y lo corre en una máquina virtual de pila. Cada objeto resuelve sus atributos y métodos recorriendo el MRO, una lista lineal de clases padre precalculada al definir la clase.

En Cruthon, el compilador agrega un paso nuevo antes de generar bytecode: construir una **tabla de pertenencia a conjuntos** por cada blob, en vez de una lista lineal de herencia. Esa tabla registra en qué blobs vive cada nombre (variable o función) y qué blobs son resultado de una intersección, unión o diferencia de otros.

```
# Bytecode actual de CPython para un atributo:
LOAD_FAST backend
LOAD_ATTR db          # busca 'db' recorriendo el MRO de backend

# Bytecode propuesto para Cruthon:
LOAD_BLOB carrito
RESOLVE_MEMBER total   # busca 'total' en la tabla de pertenencia de carrito,
                        # no en una cadena de herencia
```

`RESOLVE_MEMBER` sería el nuevo opcode central: en vez de subir por un árbol de clases, consulta directamente en qué blob(s) vive ese nombre y si el blob actual tiene acceso a él por pertenencia o por intersección. Si el nombre no está en la tabla del blob activo ni en ninguna intersección alcanzable, el intérprete lanza un error de alcance, equivalente a un `AttributeError` mal pero por razón de conjuntos, no de jerarquía.

Esto es justamente el prototipo mínimo que quedó pendiente: un fork de CPython donde `RESOLVE_MEMBER` reemplace la búsqueda por MRO para una intersección simple.

Ejemplo extendido: calculadora web con historial

Un frontend que dibuja la calculadora, un backend que hace las operaciones, y una intersección `historial` que guarda cada cálculo y permite seleccionarlo de nuevo. Usa `math` de la librería estándar de Python directamente, sin adaptación.

```
blob main:
    blob backend:
        import math

        func sumar(a, b):
            return a + b

        func potencia(base, exp):
            return math.pow(base, exp)
```

```
    blob frontend:
        func dibujar_calculadora(historial):
            render("calc.html", historial.operaciones)

        func click_operacion(indice, historial):
            return historial.operaciones[indice]

    blob historial = frontend & backend:
        operaciones = []

        func guardar(expresion, resultado):
            operaciones.append((expresion, resultado))

        func seleccionar(indice):
            return operaciones[indice]

    func calcular(a, b):
        r = backend.sumar(a, b)
        historial.guardar(f"{a} + {b}", r)
        frontend.dibujar_calculadora(historial)
        return r
```

Bytecode simplificado de la línea `historial.guardar(f"{a} + {b}", r)` dentro de `main`:

```
LOAD_BLOB historial
RESOLVE_MEMBER guardar        # busca 'guardar' en la tabla de historial
LOAD_FAST a
LOAD_FAST b
BUILD_FSTRING  "{} + {}"      # arma la expresión como texto
LOAD_FAST r
CALL_FUNCTION 2

# Dentro del propio 'guardar', al tocar 'operaciones':
LOAD_BLOB historial
RESOLVE_MEMBER operaciones     # 'operaciones' vive solo en historial, acceso
directo
LOAD_METHOD append
CALL_METHOD 1
```

Lo importante: `math.pow` corre exactamente como en Python puro, ninguna capa nueva se mete ahí. `RESOLVE_MEMBER` solo entra en juego cuando el nombre buscado pertenece a un blob con estructura de conjuntos, como `historial`. El backend nunca ve `dibujar_calculadora`, el frontend nunca ve `math`; ambos sólo comparten lo que vive dentro de `historial`.

Operadores de conjuntos: un ejemplo simple por operación

Cruthon reutiliza los operadores de sets que Python ya tiene: `&` intersección, `|` unión, `-` diferencia, `^` diferencia simétrica.

Intersección (`&`): memoria compartida entre dos blobs.

```
blob main:
    blob frontend:
        tema = "oscuro"

    blob backend:
        usuario_activo = "ana"

    blob sesion = frontend & backend:
        token = None

        func iniciar(usuario):
            token = generar_token(usuario)
```

Unión (`|`): dos blobs corren bajo el mismo proceso, sin memoria común.

```
blob main:
    blob logger:
        func registrar(msg):
            print(msg)

    blob metricas:
        func contar_evento(nombre):
            incrementar(nombre)

    blob monitoreo = logger | metricas:
        func reportar(msg, evento):
            logger.registrar(msg)
            metricas.contar_evento(evento)
```

Diferencia (`-`): lo exclusivo de un blob respecto a otro.

```
blob main:
    blob plan_premium:
        soporte_24h = True
        exportar_pdf = True
        exportar_csv = True

    blob plan_gratis:
        exportar_csv = True

    blob solo_premium = plan_premium - plan_gratis:
        # contiene soporte_24h y exportar_pdf, no exportar_csv
        func mostrar_beneficios_exclusivos():
            return ["soporte_24h", "exportar_pdf"]
```

Diferencia simétrica (`^`): lo que ninguno de los dos comparte con el otro.

```
blob main:
    blob version_a:
        boton_grande = True
        color_azul = True

    blob version_b:
        color_azul = True
        boton_redondo = True

    blob diferencias_ab = version_a ^ version_b:
        # contiene boton_grande y boton_redondo, no color_azul
        func resumen_cambios():
            return ["boton_grande", "boton_redondo"]
```

Ejemplo combinado: las cuatro operaciones juntas

Una app con dos planes de suscripción, `basico` y `pro`, mostrando cómo conviven intersección, unión, diferencia y diferencia simétrica dentro de un mismo `main`.

```
blob main:
    blob basico:
        exportar_csv = True
        soporte_email = True

    blob pro:
        exportar_csv = True
        soporte_email = True
        soporte_24h = True
```

```
        api_acceso = True

    # Intersección: lo que ambos planes comparten siempre
    blob base_comun = basico & pro:
        func funciones_garantizadas():
            return ["exportar_csv", "soporte_email"]

    # Diferencia: lo exclusivo de pro frente a basico
    blob beneficios_pro = pro - basico:
        func mostrar_upsell():
            return ["soporte_24h", "api_acceso"]

    # Diferencia simétrica: lo que distingue a cada plan del otro
    # (coincide con beneficios_pro porque basico no tiene nada propio aquí)
    blob distincion = basico ^ pro:
        func resaltar_diferencias():
            return ["soporte_24h", "api_acceso"]

    blob analitica:
        func registrar_uso(plan, funcion):
            guardar_log(plan, funcion)

    # Unión: analitica corre junto a main sin compartir memoria con los
planes
    blob sistema = analitica | base_comun:
        func iniciar_app(usuario, plan):
            analitica.registrar_uso(plan, "inicio_sesion")
            return base_comun.funciones_garantizadas()
```

Cada operación cumple un rol distinto: `base_comun` fija lo que nunca puede faltar, `beneficios_pro` aisla lo vendible del plan superior, `distincion` sirve para mostrarle al usuario justo lo que cambia entre planes, y `sistema` conecta el módulo de analytics al resto sin forzar memoria compartida con los planes.

Entorno de desarrollo: IDE visual nativo

Dado que la estructura de Cruthon es geométrica por diseño, tiene sentido que el entorno de desarrollo no sea solo texto plano, sino un editor donde cada blob se ve como un panel o burbuja con su nombre, y el programador escribe el código directamente dentro de cada uno.

La idea central: mientras se programa, el editor dibuja en vivo, a la derecha o en un panel aparte, cómo los blobs se tocan, se intersectan, se unen o se diferencian entre sí, igual que el diagrama de Venn mencionado en la sección de visualización, pero actualizándose dinámicamente mientras el código cambia, no solo como una herramienta de análisis

posterior. El programador puede hacer clic en un blob del diagrama para saltar a programar dentro de él directamente, y ver cómo el diagrama se actualiza en tiempo real a medida que escribe. Esto convertiría la relación entre blobs en algo que se ve y se manipula, no solo algo que se infiere leyendo texto, y serviría también como herramienta pedagógica para ayudar a pensar en términos de conjuntos en vez de jerarquías.

Recomendaciones de interfaz: el panel del diagrama debería poder quedar abierto permanentemente en algún lugar de la pantalla, sin ser obligatorio, con zoom y arrastre (drag) libres para examinar el sistema completo o enfocarse en una zona específica, como se navega un mapa.

Como segunda etapa del proyecto, una vez el intérprete funcione: observabilidad en tiempo real durante la ejecución, no solo durante el diseño. Ver en vivo cómo cambian las variables dentro de cada blob (las que sea posible mostrar), cómo fluye la comunicación entre blobs a través de sus intersecciones, e incluso contenido visual como imágenes si el programa las maneja. Esta capacidad de hacer evidente, mientras el programa corre, qué está pasando y dónde, podría ser el punto más fuerte de Cruthon frente a otros lenguajes, donde ese tipo de inspección normalmente requiere herramientas externas de debugging.

MCP para programación asistida

Otra línea de trabajo, independiente del IDE visual: un servidor MCP que le dé a asistentes de código como Claude Code o Codex el conocimiento de las reglas de Cruthon, para que puedan programar en el lenguaje de forma asistida sin necesitar reentrenamiento.

Esto es viable precisamente porque las reglas del lenguaje son pocas y simples: cuatro operadores de conjuntos (`&`, `|`, `-`, `^`), la regla de visibilidad geométrica (contenido visible solo si vive dentro del área de intersección o unión), y el scope resuelto por dónde vive físicamente cada función, no por quién la llama. El MCP expondría esas reglas junto con ejemplos de este documento como contexto, para que el asistente genere y corrija código Cruthon de forma consistente.

Licencia

Copyright © 2026 Luis Andrés Aguilar Luhrs. Todos los derechos reservados.

Este documento y el diseño de Cruthon que describe son propiedad exclusiva del autor. Queda prohibida su reproducción, distribución, modificación o uso, total o parcial, sin autorización expresa y por escrito del autor.